

Executive Summary

TripleA 대시보드는 **개인 투자자용 웹 데스크탑 애플리케이션**으로, 매크로 경제 지표와 사용자 계좌 현황을 한눈에 보여주고 **목표 대비 괴리 및 리밸런싱**을 지원하는 것이 목표입니다. 초기에는 AI 대신 **룰 기반 리밸런싱** 알고리즘을 적용하며, **안정성·신뢰성**을 최우선으로 개발합니다. 권장 기술 스택은 **Next.js(React) + TypeScript + Tailwind CSS** (프론트엔드), **FastAPI + SQLite/PostgreSQL** (백엔드)이며, 보안은 **JWT 토큰 기반 인증**과 **CORS 설정**을 사용합니다. ① ②. 데이터는 기존 TripleA 파이프라인(DB) 및 CSV 업로드 방식으로 공급하며, 증권사 API 연동은 후순위입니다.

본 보고서에서는 **UI 구성(화면 레이아웃 및 위젯 목록)**, **컴포넌트 트리** 및 **재사용 컴포넌트**, **API 계약(엔드포인트/요청·응답 예시)**, **DB 스키마(주요 테이블/컬럼)**, **룰 기반 리밸런싱 알고리즘 설계(수식/임계값/예시)**, **알림/경보 규칙**, **개발 일정(주별 작업, 1인·3인 기준)**, **우선순위(P0/P1/P2)**, **테스트 계획(유닛/통합/E2E)**, **배포·운영 체크리스트** 등을 상세히 제시합니다. 4주 MVP와 8주 v1 산출물을 비교하고, 1인 개발과 3인 팀의 일정·역할·리스크를 표로 정리합니다.

1. UI 구성도 (화면별 레이아웃 및 위젯)

대시보드 메인 화면을 기준으로 설명합니다. 좌측에 **사이드바(네비게이션)**, 상단에 **헤더(검색, 날짜, 알림 아이콘 등)**를 두고, 중앙 본문에 카드 형태의 패널들이 그리드 레이아웃으로 배치됩니다. 주요 위젯은 다음과 같습니다:

- **사이드바** - 대시보드, 매크로 지표, 포트폴리오, 계좌, 리포트, 자료실, 캘린더, 알림, 설정 등 메뉴.
- **헤더** - 화면 타이틀, 날짜 범위 선택, 검색창, 알림 아이콘, 사용자 프로필.
- **매크로 데이터 현황 패널** - **CPI, 금리, 환율, 실업률, VIX, PMI** 등 핵심 매크로 지표 카드(최근 추이 미니 차트 포함)와 업데이트 시간 표시. 예: “CPI YoY: 3.4% (상승)”.
- **계좌 현황 패널** - 총 평가금액, 매수 가능 금액, 계좌별 평가금액/손익/수익률, 자산군별 도넛 차트(현금·국내주식·해외주식·ETF·채권 등).
- **목표 및 괴리 현황 패널** - **설정된 목표 자산 비중**(예: 주식 60%)과 **현재 비중**을 비교하여 괴리율 표시. 경고 색상 및 게이지(bar)로 시각화. 예: “주식 목표 60%, 현재 55% (-5%, 정상)”.
- **경제 캘린더 패널** - 향후 주요 경제 일정(예: FOMC, 금통위, 실업률 발표)과 D-Day 카운트. 중요도 태그(높음/중요/보통).
- **보유 종목 Top Movers 패널** - 포트폴리오 내 종목 중 일간 등락률 상위/하위 리스트. 현재가, 등락률, 미니 스파크라인 차트.
- **리밸런싱 제안 패널** - 현재 자산 비중과 목표 비중 괴리에 따라 “비중 축소/확대/관망” 등 제안. 제안 사유(예: 목표초과, 변동성 증가) 표시.
- **알림/이상 징후 패널** - **목표 이탈 경고, 환율 급변, CPI 서프라이즈 우려, 포트폴리오 급변동** 등 알림 리스트(레벨: 정보/경고/위험). 읽음 처리 가능.
- **오늘의 인사이트 패널** - 매크로 요약, 포트폴리오 요약, 주요 이벤트 요약, 권장 대응(룰 기반). (AI 미포함 초기 버전)
- **자료실 패널** - 최근 리포트, 아이디어, 뉴스 요약 등 저장된 문서 리스트.

각 위젯은 **재사용 가능한 카드 컴포넌트**로 구성합니다. 예를 들어, 지표 카드(MetricCard), 테이블(Table), 차트(Chart) 컴포넌트를 만든 후 필요한 데이터로 템플릿화합니다.

Figure 1-2: 화면 초안 와이어프레임 예시 (데스크탑 해상도, 다크/라이트 테마)

Figure 3: 컴포넌트 트리 다이어그램 (Mermaid 스타일)

(※ 실제 화면 이미지는 참조용 와이어프레임 수준으로 설계되며, 다크 테마/라이트 테마 버전을 모두 제공합니다.)

추가 화면: 포트폴리오 상세, 계좌 상세(거래 내역), 목표 설정 페이지(자산배분/현금 비중/투자 목표 입력), 알림 목록 페이지, 자료실 검색/업로드, 캘린더 상세(주간/월간 뷰) 등을 부가적으로 설계합니다.

2. 컴포넌트 트리 및 재사용 컴포넌트

대시보드의 리액트 컴포넌트 구조 예시(간략화):

```
App
├── Layout
│   ├── Sidebar
│   ├── Header
│   └── DashboardPage
│       ├── KPIBar // 주요 KPI 요약 (총자산, 손익 등)
│       ├── MacroPanel // 매크로 데이터 카드그룹
│       │   └── MetricCard* // 지표별 카드 (재사용)
│       ├── AccountPanel // 계좌 현황 (테이블 + 차트)
│       ├── TargetPanel // 목표 대비 편차 (ProgressBar 등)
│       ├── SuggestionPanel // 리밸런싱 제안 (룰 기반 문구)
│       ├── TopMoversPanel // 상위 변동 종목 (Table)
│       ├── CalendarPanel // 경제 캘린더 (List)
│       └── InsightPanel // 오늘의 인사이트 (Text 요약)
```

- * 표시된 컴포넌트는 재사용 가능함. 예를 들어 **Card, Badge, Table, ProgressBar, MiniChart, StatusChip** 등 UI 컴포넌트로 세분화하여 범용 사용.
- **Layout, Sidebar, Header** 는 앱 전역에 걸쳐 사용.
- **DashboardPage** 하위의 패널들은 페이지별 독립 컴포넌트로 모듈화. Props 및 상태관리는 **Context**나 글로벌 상태(zustand, Redux, React Context 등)로 관리할 수 있습니다.

재사용 컴포넌트 예시:

- **Card:** 패널 레이아웃용 기본 컨테이너(그림자, 패딩).
- **Table:** 컬럼 정의와 데이터 바인딩이 가능한 범용 테이블.
- **Chart(Bar/Line/Donut):** 재사용 가능한 차트 컴포넌트(ECharts, Recharts, Chart.js 등).
- **ProgressBar:** 목표달성률이나 편차 시각화용.
- **StatusChip:** 상승/하락/경고를 색상·아이콘으로 표시.
- **Modal/Dialog:** 거래 입력, 목표 설정 팝업.
- **Button, Input, Select** 등 공용 폼 컴포넌트.

컴포넌트 트리 다이어그램은 Figure 3과 같이 구성할 수 있습니다.

3. API 계약 (엔드포인트 및 데이터 포맷)

각 패널별 데이터를 제공하는 RESTful API 엔드포인트를 설계합니다. 예시 경로와 응답 포맷은 다음과 같습니다.

- **GET /api/dashboard/summary** - 메인 대시보드 종합 정보 반환. 프론트가 한 번에 호출 가능.

```
{
  "kpi": {
    "totalAssets": 1250000,
    "cash": 300000,
    "todayProfit": 15000,
    "todayProfitRate": 1.2,
    "riskLevel": "보통"
  },
  "macro": [
    {"key": "cpi", "name": "CPI YoY", "value": 3.4, "unit": "%", "change": 0.2, "status": "rising"},
    {"key": "interest", "name": "기준금리", "value": 4.25, "unit": "%", "status": "stable"},
    // ...
  ],
  "accounts": [
    {"id": 1, "name": "종합계좌", "type": "일반", "value": 800000, "profit": 50000, "profitRate": 6.7},
    {"id": 2, "name": "주식계좌", "type": "개인", "value": 450000, "profit": 30000, "profitRate": 7.1},
    // ...
  ],
  "allocation": [
    {"asset": "Stocks", "value": 750000, "ratio": 60},
    {"asset": "Bonds", "value": 200000, "ratio": 16},
    {"asset": "Cash", "value": 300000, "ratio": 24}
  ],
  "targets": [
    {"name": "Stocks", "currentRatio": 60, "targetRatio": 65, "deviation": -5, "level": "warning"},
    {"name": "Bonds", "currentRatio": 16, "targetRatio": 15, "deviation": 1, "level": "normal"}
  ],
  "suggestions": [
    {"asset": "주식", "action": "비중 축소", "reason": "목표초과 5%"},
    {"asset": "채권", "action": "비중 확대", "reason": "목표미달 4%"}
  ],
  "topMovers": [
    {"symbol": "AAPL", "changeRate": 2.3, "contribution": 15000},
    {"symbol": "TSLA", "changeRate": -1.8, "contribution": -12000}
  ],
  "calendar": [
    {"date": "2026-05-23", "time": "21:30", "title": "미국 CPI 발표", "country": "US",
    "importance": "high"}
  ],
}
```

```

"alerts": [
  {"level": "danger", "title": "환율 급등 경고", "message": "USD/KRW 3% ↑"},
  {"level": "info", "title": "목표 비중 유지", "message": "전략적 자산 배분 이상 없음"}
],
"insights": {
  "macroSummary": "물가 상승률이 둔화되고 있으며 금리 인상 속도가 완화될 전망입니다.",
  "portfolioSummary": "총자산은 전일 대비 1.2% 상승했습니다.",
  "marketRisk": "현재 시장 위험도는 보통 수준입니다.",
  "recommendation": "비중을 다소 조정하고, 다음주 CPI 발표에 주목하십시오."
}
}

```

매개변수: 날짜 필터, 계좌 필터 등 쿼리 파라미터 선택적 지원.

- **GET /api/macro/summary** - 개별 매크로 지표 목록. (위 `dashboard` API와 동일할 수 있음)
- **GET /api/accounts** - 계좌 목록 및 요약 정보.
- **GET /api/accounts/{accountId}/positions** - 특정 계좌 보유 종목 상세.
- **POST /api/accounts/upload-csv** - 계좌 데이터(CSV) 업로드(임시).
- **GET /api/targets** - 설정된 목표 값(자산배분, 수익률 등).
- **PUT /api/targets** - 목표 수정(보통 단일 리소스/PUT).
- **GET /api/rebalancing/suggestions** - 리밸런싱 권장사항(위 예시).
- **GET /api/alerts/recent** - 최신 알림 목록. **PATCH /api/alerts/{id}/read** - 읽음 처리.
- **GET /api/calendar/events?from=&to=** - 경제 일정 조회.
- **GET /api/documents** - 자료실 문서 목록. **POST /api/documents** - 새 문서 등록 등.
- **POST /api/auth/token** - 로그인/토큰 발급 (JWT).

API 요청/응답은 JSON 형식을 사용하며, **Pydantic 스키마**를 통해 검증합니다. 타입 명세를 REST API 문서(예: OpenAPI/Swagger)로 제공합니다.

4. DB 스키마 (주요 테이블/컬럼)

SQLite(프로덕션 후 PostgreSQL로 전환 가능) 기반으로 주요 테이블을 설계합니다 ³.

- **users:** (사용자 정보)

컬럼	타입	설명
id	SERIAL PK	사용자 고유 ID
username	VARCHAR	로그인 ID
password	VARCHAR	해시된 패스워드
email	VARCHAR	이메일
created_at	DATETIME	가입 일시
- **economic_indicators:** (경제 지표)

컬럼	타입	설명
id	SERIAL PK	식별자
source	TEXT	데이터 출처 (FRED/BOK 등)
key	TEXT	지표 키 (e.g. CPI)
date	DATE	날짜
value	REAL	값
unit	TEXT	단위 (% , 포인트 등)
created_at	DATETIME	저장 일시
- **accounts:** (증권 계좌 요약)

컬럼	타입	설명
id	SERIAL PK	계좌 ID
name	TEXT	계좌명
type	TEXT	예: 종합/주식/ISA
broker	TEXT	증권사 이름
created_at	DATETIME	등록 일시

• **holdings:** (보유 종목)

| 컬럼 | 타입 | 설명 | |-----|-----|-----| | id | SERIAL PK | 식별자 | | account_id | INTEGER FK | accounts.id (소유 계좌) | | symbol | TEXT | 종목 코드 | | name | TEXT | 종목명 | | quantity | REAL | 보유수량 | | avg_price | REAL | 평균 취득 단가 | | updated_at | DATETIME | 최종 갱신 시각 |

• **targets:** (투자 목표)

| 컬럼 | 타입 | 설명 | |-----|-----|-----| | id | SERIAL PK | 식별자 | | target_type | TEXT | 예: asset_allocation, cash_ratio, monthly_invest, return_rate | | asset_class | TEXT | 주식/채권/현금/ETF/연금 등 | | target_value | REAL | 목표치(비율(%)/금액 등) | | warning_thr | REAL | 주의 임계값(%) | | danger_thr | REAL | 경고 임계값(%) | | created_at | DATETIME | 입력 일시 | | updated_at | DATETIME | 수정 일시 |

• **alerts:** (알림 로그)

| 컬럼 | 타입 | 설명 | |-----|-----|-----| | id | SERIAL PK | 식별자 | | level | TEXT | info/warning/danger | | category | TEXT | macro/portfolio/target/etc | | title | TEXT | 제목 | | message | TEXT | 내용 | | is_read | BOOLEAN | 읽음 여부 | | created_at | DATETIME | 생성 일시 |

• **documents:** (자료/리포트)

| 컬럼 | 타입 | 설명 | |-----|-----|-----| | id | SERIAL PK | 식별자 | | type | TEXT | report/idea/memo 등 | | title | TEXT | 문서 제목 | | content | TEXT | 본문 | | tags | TEXT | 태그(coma 구분) | | url | TEXT | 참조 링크(선택) | | created_at | DATETIME | 생성 일시 | | updated_at | DATETIME | 수정 일시 |

※ 실제 SQLite에서는 `check_same_thread=False` 옵션을 사용하여 멀티스레드 접근을 허용합니다 ³.

각 테이블 간 연관 관계는 필요에 따라 FK로 지정합니다. 예를 들어,

`holdings.account_id → accounts.id`, `alerts`와 `users` 테이블 연결(향후) 등을 고려합니다.

5. 룰 기반 리밸런싱 알고리즘 설계

목표 자산배분(Target)과 현재 자산구성(Current)을 비교하여 괴리를 계산하고, **임계값**을 기준으로 제안합니다. 기본 로직:

1. **비중 계산:** 각 자산군(주식, 채권, 현금 등)의 현재 비중 $\text{Curr\%} = (\text{자산군가치} / \text{총자산}) * 100$.
2. **괴리를 계산:** $\text{Dev} = \text{Curr\%} - \text{Target\%}$.
3. **임계값 설정:** 예) 주식·채권 $\pm 2\%$, 현금 $\pm 1\%$ 등 자산군별로 기준 임계값을 지정.
4. **판단:**

- $\text{Dev} > +\text{Threshold}$ → “목표초과” (과다 보유) → **비중 축소** 권장.
- $\text{Dev} < -\text{Threshold}$ → “목표미달” (과소 보유) → **비중 확대** 권장.
- $|\text{Dev}| \leq \text{Threshold}$ → “정상” → **관망**.

• **예시:**

- 목표 주식 60%, 현재 66% → $\text{Dev} = +6\%$. Threshold=3%라면 $\text{Dev} > 3\%$ → “비중 축소”(예: “주식 비중 6% 초과, 일부 매도 권장”).
- 목표 채권 15%, 현재 12% → $\text{Dev} = -3\%$. Threshold=2%라면 $\text{Dev} < -2\%$ → “비중 확대”(“채권 비중 3% 부족, 매수 권장”).

- **조정 제안 수량**(선택적): 현재 총자산 `A` 와 Dev를 이용하여 예산액 계산. 예: “주식 6% 초과 → 약 `A * 0.06` 만큼 주식 매도”.

• **추가 물:**

- **현금 비중**이 목표 미달 시 유동성 확보 권고.
- 특정 종목 편중 시(개별비중 > 목표의 N배) 경고.
- 시장 변동성 지표(VIX, 금리 등) 변화량 기반 조정 조언.

위 로직은 백엔드 서비스에서 계산하며, 필요한 입력(현재 비중, 목표, 임계값)은 API로 전달하거나 DB에서 조회합니다. 예:

```
def rebalance_suggestion(current, target, threshold):
    dev = current - target
    if dev > threshold:
        action = "비중 축소"
    elif dev < -threshold:
        action = "비중 확대"
    else:
        action = "관망"
    return action, dev
```

이때 `current` 와 `target` 은 각 자산군의 비중(%), `threshold` 는 임계치(%)입니다. 실제 반환 메시지는 한글로 “주식(현재 66%, 목표 60%) → 비중 축소 (초과 6%)”와 같이 요약합니다.

6. 알림/경보 규칙 목록

파이프라인 실행 및 데이터 모니터링 과정에서 다양한 이상 징후를 감지하여 사용자에게 알립니다. 주요 알림 규칙 예시:

- **목표 비중 이탈:** 특정 자산군 비중 괴리 > 경고 임계값 → “XX 비중 [초과/부족] 경고”.
- **현금 부족 경고:** 현금 비중 < 목표의 ‘경고 임계값’ → “현금 비중 목표 미달: 추가 현금 확보 권장”.
- **포트폴리오 급변동:** 일일손익률 > 5% 또는 누적 손실률 > X% → “포트폴리오 급등락 경고”.
- **경제 지표 변화:** 주요 지표(CPI, 기준금리 등) 예상치 대비 급변동(예: CPI 1%p 상회) → “물가 예상 초과 변동”.
- **미래 일정 주의:** 중요 일정(예: 금통위, FOMC) D-1일 전 알림 → “내일 FOMC 예정”.
- **데이터 수집 실패:** 수집 스케줄러 오류 또는 특정 지표 fetch 실패 시 로그 알림 → “FED 데이터 수집 실패: 네트워크 오류”.
- **보고서/문서 공유:** 신규 리포트 생성 시 알림(필요 시).

알림은 `alerts` 테이블에 저장되고, 사용자가 확인(읽음)하면 `is_read` 가 갱신됩니다. **우선순위(level)**는 통상 “info/normal”, “warning”, “danger”로 구분합니다.

7. 개발 타임라인 및 인원 배분

7.1. 8주 개발 계획 (1인 기준)

혼자 개발할 경우 주차별 목표를 명확히 설정하고 순차적으로 진행합니다. 예시 일정:

주차	목표	주요 작업 (1인 개발 기준)	산출물
1주차	UI 골격·레이아웃 구축	Next.js 프로젝트 셋업, Tailwind 설정, 레이아웃 컴포넌트 (사이드바/헤더/카드) 작성, 대시보드 페이지 기초 구현	대시보드 초안 (모크 데이터 사용)
2주차	매크로·KPI 패널 개발	KPI 카드 구현, 매크로 데이터 패널 컴포넌트, 미니 차트(라인/스파크라인) 포함, 모크 데이터로 검증	매크로 지표 패널 완성
3주차	계좌·자산배분 패널 개발	계좌 요약 카드 구현, 자산배분 도넛 차트, 계좌 목록 테이블, 모크 API 작성	계좌 현황 패널 완성
4주차	목표 괴리·알림 패널 개발	목표 데이터 모델 설계, 괴리율 계산 컴포넌트, 진행률 바 구현, 알림 리스트 UI	목표 대비 패널, 알림 UI 완성
5주차	캘린더·Top Movers·자료 패널	경제 캘린더 리스트 구현, 이벤트 태그, Top Movers 테이블, 자료실 요약 컴포넌트	캘린더/Top Movers/자료 패널 완성
6주차	리밸런싱 제안·인사이트	리밸런싱 로직 구현(룰 기반), 제안 카드, 오늘의 인사이트 UI, 모크 데이터 로직 테스트	제안·인사이트 패널 완성
7주차	백엔드 API 구현·통합	FastAPI 서버 구축, 앞서 정의된 엔드포인트 구현, DB(SQLite) 연동, CORS 설정 ²	API 서버 (Swagger 문서 포함)
8주차	실데이터 연동·QA	기존 TripleA DB 연결 (economic_data.db), 수집 파이프라인과 통합, 오류 수정 및 최종 QA, README 작성	v1.0 배포 준비 (로컬 환경 완성)

7.2. 3인 팀 개발 기준

3인 팀이라면 **역할 분담**과 **병렬 작업**을 통해 속도를 높입니다. 예:

- **프론트엔드 개발자(1명)**: UI/UX 설계·구현, React 컴포넌트, 스타일링, API 연동(클라이언트).
- **백엔드 개발자(1명)**: FastAPI 서버, DB 모델링/연동, API 개발, 데이터 파이프라인 연결.
- **데브옵스/QA(1명)**: 배포 파이프라인 구성, 테스트 작성(E2E 포함), 인프라 모니터링/CI 구축, 문서화 지원.

3인 팀 시 일정 예:

주차	프론트엔드	백엔드	QA/인프라
1주차	Layout/UI 컴포넌트 베이스 제작	FastAPI 프로젝트 환경 구축	리포지토리 셋업, CI/CD 기초 구성
2주차	KPI·매크로 패널 구현	SQLite DB 스키마 설계/적용	유닛 테스트 설정, 로깅 정책 수립
3주차	계좌·자산 배분 패널 구현	계좌·보유종목 API 개발	인테그레이션 테스트 작성 (단계별)

주차	프론트엔드	백엔드	QA/인프라
4주차	목표/괴리 패널 구현	목표 API, 알림 로직 개발	시스템 테스트, 보안검토
5주차	캘린더/Top Movers/자료 패널 구현	이벤트·자료 API 개발	슬랙/메일 알림 연동 검토
6주차	리밸런싱 제안·인사이드 UI 구현	룰 기반 리밸런싱 서비스 개발	성능 테스트, 부하테스트
7주차	API 연동·프론트엔드 최종 조율	실데이터 파이프라인 통합	QA 완료 및 도커화(선택)
8주차	버그 수정·UI 개선	배포 스크립트 작성·API 최적화	모니터링 설정·문서화 (운영 매뉴얼)

역할별 위험(Risk): 1인 개발은 일정 지연 시 전체 프로젝트 지연 위험이 크고, 코드 리뷰 어려움이 있습니다. 3인 팀은 커뮤니케이션 비용과 합의 불일치(merge conflict, 기준 정의 등) 위험이 있지만, 작업 병렬화로 일정 준수가 유리합니다.

<u>개발 비교: 1인 vs 3인</u> (예시)

구분	1인 개발	3인 팀 개발
일정	순차적 진행 (8주)	병렬 개발 가능 (대략 5~6주)
역할	Full-stack (UI+API+DB+QA)	프론트, 백엔드, QA/운영 분담
리스크	작업량 과중, 병목 위험 큼	협업 조정·병합 충돌 위험
의사결정	빠름 (개인이 독단)	느림(회의 필요)
코드 품질	개인 역량 의존	코드 리뷰·검증 가능 (견고)

7.3. 4주 MVP vs 8주 v1 산출물 비교

MVP는 핵심 기능만 구현하여 빠르게 출시합니다. 4주 vs 8주 산출물을 표로 비교하면 다음과 같습니다.

항목	4주 MVP	8주 완성 (v1.0)
UI/UX	대시보드 레이아웃, 카드 컴포넌트	추가 화면(목표 설정, 자료실 등)
매크로 데이터	주요 지표 카드 3~5개	추가 지표(환율, VIX, PMI 등)
계좌/포트폴리오	총자산, 계좌별 요약	자산배분 도넛, Top Movers 추가
목표 관리	목표 대비 괴리 표시	목표 설정 UI, 알림 레벨 세분화
리밸런싱	간단 룰 기반 제안	제안 사유 상세화, 경고 강화
자동화/연동	수동 실행, 모크 데이터	FastAPI 연동, DB 저장, CORS 적용
테스트/문서	최소한의 테스트	유닛/통합/엔드투엔드 테스트, README
제외 항목	캘린더 상세, PDF 업로드, AI 리포트 등	제거 항목 구현

즉, 4주 MVP에서는 대시보드 기본 뼈대, 주요 패널, 룰 기반 리밸런싱 제안까지 완성하고, 8주 v1에서는 자동화 연동, 알림, 테스트/문서화, 안정성을 추가 완료하는 것을 목표로 합니다.

8. 우선순위 (P0/P1/P2)

• P0 (필수):

- 기본 실행 - `python main.py` 한 번 실행으로 데이터 수집→저장→차트/리포트 생성 및 로그 출력이 되는 구조 ².
- UI/UX - 대시보드 레이아웃, 카드 컴포넌트, 사이드바/헤더, 다크모드 적용.
- 매크로 데이터 - CPI, 금리, 환율 등 핵심 지표 수집 및 패널 표시.
- 계좌/자산배분 - 총자산, 계좌 요약, 도넛 차트.
- 목표/괴리 계산 - 목표 대비 비중 계산·표시, 임계값 경고.
- 리밸런싱 제안 (룰 기반) - 비중 편차 분석 후 권장 액션 도출.
- API & DB 연동 - SQLite 스키마, 주요 API 엔드포인트 구축, CORS 활성화 ².
- 기본 로깅 - 수집/실행 로그 기록, 실패 원인 파악 로그.

• P1 (우선 구현):

- 경제 캘린더 - 글로벌/국내 주요 일정 리스트.
- Top Movers - 보유 종목 등락순 정렬 목록.
- 알림/경보 - 알림 카운트, 읽음 처리, 중요도별 UI.
- 자료실 - 리포트/아이디어/뉴스 CRUD UI.
- 목표 설정 UI - 사용자가 목표 비중 등 입력·수정.
- CSV 기반 계좌 데이터 입력 - 증권사 API 대신 CSV 업로드.
- 기본 테스트 작성 - 핵심 함수 유닛 테스트 (날짜 파싱, 리밸런싱 로직 등).

• P2 (추후 확장):

- 실시간 시세 연동 - 주식/환율 실시간 API.
- 증권사 API 연동 - OAuth 기반 계좌/주문 API.
- AI 리포트 - Gemini(LLM) 프롬프트 개선 자동화.
- Docker 배포/클라우드 배포 - 컨테이너화, AWS/GCP 배포.
- Slack/Email 알림 - 장애·경고 시 외부 알림 연동.
- 백테스트 기능 - 투자 전략 테스트 모듈.

9. 테스트 계획

9.1. 유닛 테스트 (Unit Test)

- 백엔드 로직: 날짜 파싱, 리밸런싱 계산, 데이터 검증 함수, DB upsert 기능 등.
- 프론트엔드 유틸: 데이터 포맷 변환, 시각화(차트 데이터) 함수.
- 컴포넌트 동작: 주요 컴포넌트 렌더링 및 이벤트(버튼 클릭 등) 단위 테스트(Jest, React Testing Library).

9.2. 통합 테스트 (Integration Test)

- **API 테스트:** FastAPI 테스트 클라이언트를 통한 엔드포인트 호출, DB와 연동된 경로 검증 (pytest + SQLAlchemy).
- **데이터 파이프라인:** 실제 경제 데이터 수집→DB 저장→API 제공까지 연결 검증.
- **프론트엔드-백엔드 연동:** TanStack Query를 이용한 API 콜 테스트(예: MSW 활용).

9.3. E2E 테스트 (End-to-End)

- **시나리오 테스트:** Cypress/Puppeteer 등으로 주요 사용자 흐름 자동화 테스트. 예: 로그인(토큰 발급) → 대시보드 조회 → 목표 변경 → 리밸런싱 제안 확인 → 로그아웃.
- **예러 처리:** 백엔드 오류나 네트워크 단절 시 프론트엔드 UI 동작(로딩, 오류 메시지 표시) 테스트.

모든 단계에서 **커버리지**를 확인하고, CI 파이프라인(GitHub Actions 등)에 테스트 스크립트를 포함하여 자동화합니다.

10. 배포·운영 체크리스트

- **로그 관리:**
 - 애플리케이션 로그: 수집/처리 과정을 로깅(성공/실패 이벤트, 소요시간 등).
 - 오류 모니터링: 예외 처리 시 스택 트레이스 저장.
- Log Aggregation: Sentry, ELK, Rollbar 같은 도구 고려.

- **모니터링:**
 - 시스템 상태: 서버 CPU/RAM 사용량, 디스크 공간 감시.
 - 애플리케이션 성능: 응답 시간, 에러율, 처리 속도(지표 업데이트 간격) 체크.
 - Heartbeat: 정기 실행(launchd/크론) 성공 여부 알림.

- **보안:**
 - 인증/인가: JWT 토큰 만료, 비밀번호 안전 저장(Argon2/Bcrypt) ⁴ ⁵.
 - CORS 설정: 허용 도메인 명시 (React 앱 호스트) ².
 - HTTPS: 배포 시 TLS 적용.

- **데이터 백업:**
 - 정기 백업: DB 파일 또는 덤프를 주기적(일간/주간) 백업 스토리지에 저장.
 - 버전 관리: 마이그레이션 스크립트 관리 및 스키마 변경시 롤백 전략.

- **운영 자동화:**
 - 런치 스케줄: macOS `launchd`, Linux `cron` 또는 **GitHub Actions** 워크플로우로 주기 실행 설정.
 - 최신 상태 요약: 실행 결과를 `outputs/latest_status.json`로 기록(성공/부분 실패 등).
 - 알림 전송: 필요 시 슬랙/메일/텔레그램 연동을 차후 고려.

이 체크리스트를 기반으로 배포 전에 **보안 점검 리스트**와 **운영 매뉴얼**을 문서화하여 리스크를 최소화합니다.

표: 비교 및 요약

1인 개발 vs 3인 팀 (일정·역할·리스크 비교)

구분	1인 개발	3인 팀
일정	8주(순차적)	~6주(병렬 작업)
역할	전원 Full-stack (Frontend+Backend)	프론트엔드 1명, 백엔드 1명, QA/DevOps 1명
진행 방식	단일 승인, 빠른 결정	정기 미팅, 합의 필요
리스크	지연시 프로젝트 전체 일정 지연, 코드 리뷰 부족	커뮤니케이션 비용, merge 충돌 가능
장점	간단한 관리, 의사 결정 신속	전문 역할 분담, 높은 생산성 가능

4주 MVP vs 8주 v1 산출물 비교

범주	4주 MVP	8주 v1.0
화면/UI	대시보드 기본 화면(모크 데이터)	전체 화면(목표설정, 계좌상세 등 추가)
주요 기능	매크로 지표 표시, 계좌 요약, 목표 괴리 표시, 룰 기반 리밸런싱	추가 매크로 지표, 리밸런싱 강화, 알림 시스템
데이터	모크 데이터 / CSV 임시 입력	FastAPI-DB 연동, 실제 수집 파이프라인 사용
자동화	수동 실행	launchd/cron 자동 실행 설정
문서/테스트	최소 문서, 제한적 테스트	README, API 문서, 유닛·통합·E2E 테스트
제외 기능	경제 캘린더 상세, AI 리포트, 증권사 연동 등	포함하여 완전한 V1 구현

참고 자료: Next.js는 React 기반 풀스택 웹 프레임워크로, 빌드 설정을 자동화하여 개발 속도를 높여 준다 ¹. Tailwind CSS는 HTML 내 유틸리티 클래스 방식의 빠른 UI 구현을 지원한다 ⁶. FastAPI는 Pydantic과 SQLAlchemy 기반으로 SQL(예: SQLite) 연동이 용이하며, CORS 미들웨어를 통해 프론트엔드 요청을 안전하게 처리할 수 있다 ² ³.

각 단계별 산출물과 완료 기준을 명확히 하여 **안정적인 데이터 파이프라인 구축**과 **사용자 경험 개선**을 동시에 달성하는 것이 개발의 핵심입니다.

¹ Next.js Docs | Next.js

<https://nextjs.org/docs>

² CORS (Cross-Origin Resource Sharing) - FastAPI

<https://fastapi.tiangolo.com/tutorial/cors/>

³ SQL (Relational) Databases - FastAPI

<https://fastapi.tiangolo.com/tutorial/sql-databases/>

4 5 OAuth2 with Password (and hashing), Bearer with JWT tokens - FastAPI

<https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/>

6 Tailwind CSS - Rapidly build modern websites without ever leaving your HTML.

<https://tailwindcss.com/>